



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G1

System Integration and Architecture

System Design Document (SDD)

Project Title: ByteZone

Prepared By: Ursulo, Lichael Yashua E.

Version: 0.3

Date: 5/27/2026

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	02/21/2026	Lichael Yashua E. Ursulo	Initial draft	Draft
0.2	02/28/2026	Lichael Yashua E. Ursulo	Added contents for System Architecture, API Contract and Communication Flow, Database Design, UI/UX Design, and Project Plan	Final
0.3	05/26/2026	Lichael Yashua E. Ursulo	Updated final implementation details including Supabase PostgreSQL database hosting, Render backend deployment, Vercel web deployment, Cloudinary profile image upload, RAWG public API integration, Google OAuth implementation for web and Android, SMTP email configuration, Android Kotlin mobile app completion, payment sandbox flow, and added architecture views for client-server, layered, REST API, and MVC-style design pattern.	Final Implementat ion Update

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	7
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	7
2.1 Project Overview	7
2.2 Core User Journeys	7
2.3 Feature List (MoSCoW)	8
2.4 Detailed Feature Specifications	10
2.5 Acceptance Criteria	14
3.0 NON-FUNCTIONAL REQUIREMENTS	17
3.1 Performance Requirements	17
3.2 Security Requirements	17
3.3 Compatibility Requirements	18
3.4 Usability Requirements	18
4.0 SYSTEM ARCHITECTURE	18
4.1 Component Diagram	18
4.2 Client-Server Architecture	19
4.3 Layered Architecture	20
4.4 REST API Architecture	21
4.5 Applicable Design Pattern	21
4.6 Technology Stack	22
5.0 API CONTRACT & COMMUNICATION	22
5.1 API Standards	22
5.2 Endpoint Specifications	23
5.3 Error Handling	36
6.0 DATABASE DESIGN	38
6.1 Entity Relationship Diagram	38
6.2 Detailed Relationships	39
6.3 Key Tables	39
6.4 Table Structure Summary	39
7.0 UI/UX DESIGN	40
8.0 PLAN	50

EXECUTIVE SUMMARY

1.1 Project Overview

ByteZone is a computer café management system that helps cafés handle customer accounts, station usage, station booking/reservations, session timing (ByteZone acts as the timer), snack ordering, announcements, and transaction tracking in one platform. Instead of relying on manual tracking and disconnected tools, ByteZone allows customers to book stations and order snacks, while staff/admin can control sessions, manage orders, publish announcements, and monitor transactions through dashboards.

The final implementation of ByteZone uses a deployed full-stack architecture composed of a ReactJS web application, Android Kotlin mobile application, Spring Boot backend API, and Supabase PostgreSQL database. The backend is deployed on Render, the web frontend is deployed on Vercel, and the mobile app connects to the same deployed backend through Retrofit. The system also integrates Google OAuth, Cloudinary profile image upload, SMTP email configuration, RAWG public game data, and a sandbox payment flow for testing payment-related features.

1.2 Objectives

1. Allow users to create an account, log in, view a personal dashboard or profile, and log out securely.
2. Ensure protected pages such as the dashboard and profile cannot be accessed when the user is logged out.
3. Implement role-based access with three roles: User, Staff, and Admin, each with different permissions.
4. Provide ByteZone's core feature of built-in station session timing where Staff and Admin can start sessions, extend time, and end sessions per station.
5. Provide a station booking and reservation feature that allows users to reserve a station following booking rules.
6. Provide a snack ordering feature that allows users to place orders and allows Staff and Admin to manage the order status.
7. Record payment options using Payment Gateway Integration Sandbox

8. Provide Staff and Admin management features including announcements, user management with role assignment, and transaction history for sessions, extensions, and snack orders.
9. Implement required integrations including public API information displayed in the system, Google login with the system issuing its own login token, file upload linked to database records, and email sending for account and notification purposes.

1.3 Scope

Included Features:

- Account creation, login, dashboard or profile access, and logout
- Protected pages that cannot be opened when logged out
- Role-based access using three roles: User, Staff, and Admin
- Built-in station session timing where Staff and Admin can start sessions, extend time, and end sessions per station
- Station booking and reservation where users can reserve a station based on defined booking rules
- Snack ordering where users can place orders and Staff and Admin can manage order status updates
- Payment recording using Payment Gateway Integration Sandbox with payment status tracking as Pending or Paid and Staff or Admin confirmation
- Admin and Staff dashboard showing total users, new users, active users, announcements, transactions, and pending payments
- User dashboard showing player statistics, current session details, reservation status, quick actions, and ByteZone updates
- User management where Admin can edit and delete users and assign roles as User, Staff, or Admin
- Tournament wins tracking where Staff and Admin can update a user's tournament wins record

- Announcements management where Staff and Admin can create, edit, and delete announcements
- Orders management where Staff and Admin can review orders and update order progress until completion
- Transaction history showing all session extensions and snack order transactions
- Web frontend deployed through Vercel and connected to the deployed backend API
- Backend API deployed through Render and connected to Supabase PostgreSQL
- Android Kotlin mobile app for User role features only, using Retrofit and the same backend API
- Google OAuth login implemented for web and Android mobile, with backend token verification and ByteZone JWT generation
- Cloudinary-based profile image upload with uploaded image URL linked to user profile records
- RAWG public API integration for Top Picks for Gamers / gaming highlights
- SMTP email configuration for account and notification-related email support
- Sandbox payment checkout for reservation payments, snack orders, and session extension payments
- Notification updates for reservations, payments, and snack order status changes

Excluded Features:

- Real payment processing through a payment gateway
- Automatic computer locking and unlocking controlled by the system
- Multi-branch support for multiple café locations
- Advanced AI optimization

1.0 INTRODUCTION

1.1 Purpose

This document is the Software Design Document (SDD) for ByteZone. It provides the complete design blueprint, functional and non-functional requirements, architecture, API contracts, database design, UI/UX plan, and project plan to guide implementation and ensure all components integrate correctly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: ByteZone

Domain: Computer Café Management System

Primary Users:

1. Customer/Player
2. Staff
3. Admin

Problem Statement: Computer cafés experience delays and confusion when station usage timing, reservations, snack ordering, and payments are handled manually or through disconnected tools. During peak hours, staff struggle to monitor which stations are occupied, how much time remains, who has reservations, and which orders are pending, affecting service quality and operational efficiency.

Solution: ByteZone centralizes station reservations, session timing, snack ordering, announcements, and transaction tracking into a secure role-based system accessible through web and mobile apps.

2.2 Core User Journeys

Journey 1: Register → Login → Dashboard (User)

1. User opens ByteZone
2. User registers
3. User logs in
4. User is redirected to the dashboard

5. If the user logs out, the dashboard can no longer be accessed

Journey 2: Book Station Reservation (User)

1. User opens "Book" page
2. User selects date/time and station
3. System validates booking rules (1 booking/day; can book next day)
4. System checks availability / Staff approves / Reservation status updated.
5. System saves reservation then status will be shown in dashboard

Journey 3: Staff Session Timing (ByteZone Timer)

1. Staff logs in
2. Staff starts a session on a station
3. System shows remaining time and station status "In Use"
4. Staff extends time upon request (creates a pending transaction if unpaid)
5. Staff ends session then the station returns "Available"

Journey 4: Snack Ordering (User → Staff)

1. User browses snacks in "Snacks" page and places order
2. Staff sees order list
3. Staff updates status: Preparing → Ready → Served
4. User receives status notification/popup

Journey 5: Payment Confirmation (Staff/Admin)

1. Transaction appears as Pending
2. Staff/admin confirms payment at counter or through online checking
3. System marks transaction Paid and includes it in history

Journey 6: Logout

1. User selects Logout
2. System ends the user's login session
3. User is returned to the login page and cannot open protected pages

2.3 Feature List (MoSCoW)

MUST HAVE

1. Authentication (register, login, logout, protected pages, current-user endpoint)
2. Role-based access (User, Staff, Admin; API + UI restriction)
3. Core modules with CRUD + validation (non-User entities)
4. User dashboard insights (favorite game, last played, hours played)
5. Admin/Staff dashboard metrics (active users, new users, pending payments)
6. Station timer/session management (start/extend/end, station status)
7. Station booking/reservation feature with rules (1 booking/day; next day booking)
8. Snack ordering + staff status workflow (Preparing/Ready/Served)
9. Notifications UI panel for order and payment updates
10. Announcements management
11. User management (admin role changes; tournament wins field update)
12. Transaction history (sessions/extensions/snack orders)
13. External public API integration
14. Google OAuth login + link to DB + generate own token
15. File upload + store on server + link to DB + view/download
16. Email sending via SMTP (account + notification)
17. Mobile app (Android Kotlin XML, API 34, Retrofit, same backend, no mock data)
18. Minimum 5 DB tables with proper relationships, DTOs, no plaintext passwords
19. Payment recording using Payment Gateway Integration Sandbox

SHOULD HAVE

1. Search/filter stations (Available/In Use)
2. Search/filter snack menu and orders
3. Order history and session history for users
4. Clear validation messages and error feedback
5. Responsive UI for mobile and desktop

COULD HAVE

1. Basic dashboard analytics (most used station, peak hours, top snacks)
2. Real-time updates (WebSocket or polling) bonus

WON'T HAVE

1. Automatic PC lock/unlock control from the system
2. Advanced AI forecasting/optimization features (future enhancement)

3. Multi-branch support with location-based management

2.4 Detailed Feature Specifications

Feature: User Authentication & Access Control

- **Screens:** Registration, Login, Dashboard, Profile, Logout
- **Fields:** Fullname, Email, Password
- **Validation:** Required fields, email format, fullname/email uniqueness, password stored securely
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout, GET /me
- **Security:** Session token (JWT), password hashing (BCrypt), protected routes/pages

Feature: Station Booking

- **Screens:** Book Station, Reservation Status, Staff Reservation List
- **Display:** Station number, station status, available stations, selected date and time, reservation status, and Booking Summary checkout
- **Functions:** View available stations, choose date and time, book a station, view reservation status, cancel reservation, staff can approve and check-in reservations
- **Reservation Status Flow:** Pending → Approved → Checked In → Completed, Cancelled, Expired
- **Rules:** One booking per user per day, booking allowed for the next day, station must be available for the selected time slot
- **Validation:** Date and time must be valid, station must be available, user must not exceed one booking per day
- **API Endpoints:** GET /stations, POST /reservations, GET /reservations/me, GET /reservations, PUT /reservations/{id}
- **Access Control:** Customer can create and view their own reservations; Staff and Admin can view all reservations and update status

Feature: Snack Ordering & Order Tracking

- **Screens:** User Snack Menu, Order Summary checkout, User Order Status Popup/Panel, Staff Orders Page
- **Display:** Snack name, price, availability, quantity, order total, current order status
- **Functions:** Browse snacks, add to cart, place order, view order status, view order history, staff can accept orders and update progress until completion

- **Order Status Flow:** Pending → Preparing → Ready → Served, Cancelled
- **Validation:** Quantity must be greater than zero, item must be available, cart must not be empty
- **API Endpoints:** GET /snacks, POST /orders, GET /orders/me, GET /orders, PUT /orders/{id}/status
- **Access Control:** Customer can place orders and view their own orders; Staff and Admin can view all orders and update order status

Feature: Payment Gateway Integration and Payment Tracking

- **Screens:** Payment Options, Payment Checkout, Payment Result, Staff Payments Panel
- **Display:** Amount to pay, payment method options, payment status, payment reference number, transaction details
- **Functions:** Choose payment method, proceed to sandbox checkout, confirm payment result, record payment details, view payment history, staff can view all payments and statuses
- **Payment Status Flow:** Initiated → Processing → Paid, Failed, Cancelled, Refunded
- **Validation:** Amount must be valid, transaction must be linked to a session extension or snack order, payment cannot be marked paid without gateway confirmation
- **API Endpoints:** POST /payments/initiate, POST /payments/webhook, GET /payments/me, GET /payments, GET /payments/{id}
- **Access Control:** Customer can initiate payment and view their own payment records; Staff and Admin can view all payments and verify payment status)

Feature: Payment Gateway Integration Sandbox

- **Screens:** Sandbox Checkout, Payment Result, Pending Payments, Transaction History
- **Display:** Payment ID, payment type, amount, reference number, payment status, and sandbox QR-style display
- **Functions:** Create sandbox payment, simulate Pay Success, simulate Fail Payment, cancel payment, update payment status, record transaction, and notify user
- **Payment Types:** RESERVATION, SNACK_ORDER, SESSION_EXTENSION
- **Payment Status Flow:** INITIATED → PROCESSING → PAID / FAILED / CANCELLED

- **Validation:** Payment must be linked to a valid reservation, snack order, or session extension; payment cannot be updated if it does not exist
- **API Endpoints:** POST /api/payments/initiate, POST /api/payments/sandbox/{id}/success, POST /api/payments/sandbox/{id}/fail, POST /api/payments/sandbox/{id}/cancel, GET /api/payments/me, GET /api/payments, GET /api/payments/pending
- **Note:** This is a sandbox simulation only and does not process real money

Feature: Admin and Staff Dashboard and Management

- **Screens:** Dashboard, User Management, Announcements, Extending Time, Snacks, Orders, Transaction History, Pending Payments
- **Display:** User counts, announcements, transactions summary, pending payments, user roles, tournament wins, active sessions, orders list, order status
- **Functions:** View metrics, manage users and roles, update tournament wins, create and manage announcements, extend station time, manage orders with status updates, confirm pending payments, view transaction history
- **Validation:** Admin only role assignment, extension time must be valid, station must be active to extend, order status must follow flow, payment must exist before confirming
- **API Endpoints:** GET /admin/metrics, GET /admin/users, PUT /admin/users/{id}, DELETE /admin/users/{id}, GET /announcements, POST /announcements, PUT /announcements/{id}, DELETE /announcements/{id}, PUT /sessions/{id}/extend, GET /orders, PUT /orders/{id}/status, GET /transactions, GET /payments, PUT /payments/{id}/confirm
- **Access Control:** Staff manages operations; Admin manages operations and users/roles

Feature: Google Login

- **Screens:** Login Page, Account Linking Prompt, Dashboard Redirect
- **Display:** Google login button, basic profile details after login, login success message
- **Functions:** Log in using Google, create or link account to an existing user record, redirect user to dashboard after successful login
- **Validation:** Only allow verified Google login, prevent duplicate accounts by matching email, require role assignment default to User
- **API Endpoints:** POST /auth/google
- **Access Control:** Available for all users; role permissions still apply after login

Feature: Android Mobile User App

- **Screens:** Login, Register, Dashboard, Book Station, Order Snacks, Sandbox Checkout, Notifications, Transaction History, and Profile
- **Display:** User dashboard information, current session, remaining time, reservation status, snack list, order summary, payment status, notifications, and profile details
- **Functions:** User login, Google login, dashboard viewing, station booking, snack ordering, sandbox payment processing, notification viewing, transaction history viewing, profile update, and profile image upload
- **Validation:** User must be authenticated, Staff/Admin accounts are blocked from mobile access, required fields are validated, and network/API errors are handled with proper messages
- **API Communication:** Retrofit and OkHttp are used to connect the mobile app to the deployed Render backend
- **Access Control:** Mobile app is limited to User role accounts only

Feature: Profile Image Upload Linked to User Record

- **Screens:** Account Information / Profile Page
- **Display:** Current profile image, uploaded image preview, profile details, and update confirmation
- **Functions:** Upload profile image, update profile image, display uploaded profile image in dashboard/profile, store uploaded image externally, and save the image URL in the user record
- **Validation:** File must be an accepted image type, file must not exceed allowed size, user must be authenticated before uploading
- **API Endpoints:** POST /api/user/me/profile-image, PUT /api/user/me
- **Storage Integration:** Cloudinary is used as the external image storage provider
- **Access Control:** Users can upload and update their own profile image; Staff/Admin accounts can still be managed through the web dashboard

Feature: Email Notifications

- **Screens:** Email is delivered to user inbox, Notification trigger shown in system logs
- **Display:** Email subject, message body, timestamp of sending
- **Functions:** Send account-related email after registration, send notification email when order status becomes ready or when payment is confirmed
- **Validation:** Email must be valid, sending retries on failure, do not send duplicates for the same event

- **API Endpoints:** Email is triggered by backend service events such as registration and status updates
- **Access Control:** Users receive emails for their own account; Staff and Admin actions can trigger notification emails

2.5 Acceptance Criteria

AC-1 Successful User Registration

- Given I am a new user
- When I enter a valid fullname, email, and password
- And I submit the registration form
- Then my account should be created successfully
- And I should be able to log in using my credentials

AC-2 Successful User Login and Protected Access

- Given I have a registered account
- When I enter correct login credentials
- Then I should be logged in successfully
- And I should be redirected to my dashboard or profile
- And protected pages should be accessible

AC-3 Block Protected Pages When Logged Out

- Given I am not logged in
- When I try to open the dashboard or any protected page
- Then I should be redirected to the login page
- And the protected content should not be displayed

AC-4 Station Booking Success

- Given I am logged in as a user
- When I select a valid date and time and choose an available station
- And I confirm the booking
- Then the reservation should be created
- And my reservation details should appear in my reservation status section

AC-5 One Booking Per Day Rule

- Given I already have a reservation for today
- When I attempt to create another reservation for the same day
- Then the system should block the booking
- And show an error message explaining the one booking per day rule

AC-6 Staff Reservation Status Update

- Given I am logged in as staff or admin
- When I open the reservation list
- And I approve or check in a reservation
- Then the reservation status should update correctly
- And the user should see the updated reservation status on their dashboard

AC-7 Start Station Session Timer

- Given I am logged in as staff or admin
- When I select an available station and start a session
- Then the station status should change to in use
- And the session should show a start time and remaining time
- And the session should appear in the active sessions list

AC-8 Extend Playtime Creates Pending Payment

- Given there is an active session on a station
- When staff or admin extends the session by adding minutes or hours
- Then the session end time should be updated
- And a transaction should be recorded
- And the payment status should be set to pending

AC-9 Pending Payment Visibility and Reminder

- Given a session extension or snack order has a pending payment
- When the user views their dashboard
- Then the system should show a payment reminder message
- And when staff or admin views pending payments
- Then the unpaid user and transaction should appear in the pending payments list

AC-10 Confirm Payment and Update Status

- Given there is a pending transaction
- When staff or admin confirms payment
- Then the payment status should change to paid
- And the transaction should be reflected in the transaction history
- And the pending payments list should update accordingly

AC-11 Place Snack Order

- Given I am logged in as a user
- When I select snack items with valid quantities
- And I place the order

- Then the order should be saved successfully
- And the order should appear in my order status and order history

AC-12 Snack Order Status Updates and User Notification

- Given I placed a snack order
- When staff or admin changes the order status to preparing
- Then I should see a notification that my food is being prepared
- And when staff or admin changes the order status to ready
- Then I should see a notification that my food is ready and will be served
- And when staff or admin marks the order as served
- Then the final status should be shown in my order history

AC-13 Admin User Role Management

- Given I am logged in as an admin
- When I change a user's role to staff or admin
- Then the user's role should update successfully
- And the updated role should take effect on the next login
- And the user should only see pages allowed by their role

AC-14 Tournament Wins Update

- Given I am logged in as staff or admin
- When I update a user's tournament wins to a valid value
- Then the new tournament wins count should be saved
- And it should appear on the user's dashboard status section

AC-15 Announcements Management and Visibility

- Given I am logged in as staff or admin
- When I create or edit an announcement and save it
- Then the announcement should appear in the announcements list
- And users should see it in the ByteZone updates section on their dashboard

AC-16 Google Login Success

- Given I have a Google account
- When I sign in using Google login
- Then my account should be created or linked
- And I should be logged in and redirected to the dashboard

AC-17 File Upload Linked to Record

- Given I am logged in
- When I upload a file and attach it to a valid record
- Then the file should be stored successfully
- And I should be able to view or download it later

AC-18 Account Email Sent

- Given I successfully register an account
- When registration is completed
- Then the system should send an account-related email to my email address

AC-19 Notification Email Sent

- Given I have a snack order or payment confirmation event
- When staff marks the order as ready or confirms payment
- Then the system should send a notification email to my email address

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time ≤ 2 seconds for most requests
- Station list/session data refresh quickly for staff operations
- Database queries optimized (indexes on reservations, sessions, orders)
- Render free-tier backend may experience cold starts, so the first request after inactivity may take longer than normal
- Mobile and web clients should handle loading states while waiting for API responses

3.2 Security Requirements

- HTTPS required
- Passwords stored hashed (no plaintext)
- Token-based authentication; protected routes/pages
- Role checks on restricted endpoints
- DTOs never expose password fields
- Environment variables are used for sensitive values such as database credentials, JWT secret, Google Client ID, Clouinary credentials, SMTP credentials, and RAWG API key
- Google OAuth ID tokens are verified by the backend before issuing ByteZone's own JWT token
- Mobile API requests attach the JWT token through an authorization interceptor
- Staff/Admin accounts are blocked from using the Android mobile app and are redirected to the web dashboard

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** Android 14 / API 34 target, Kotlin, XML UI, Retrofit, OkHttp, Google Sign-In
- **Deployment Compatibility:** Backend deployed on Render, web deployed on Vercel, database hosted on Supabase PostgreSQL

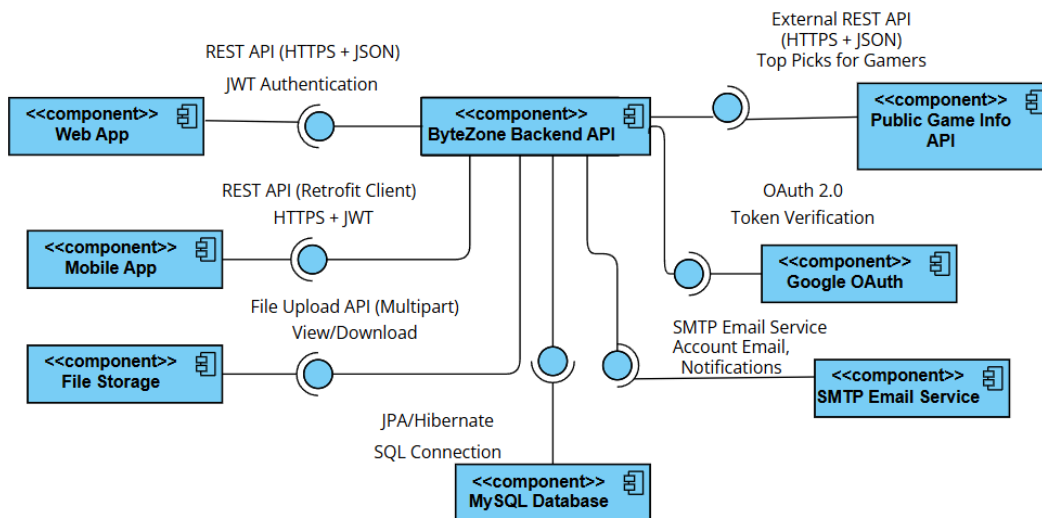
3.4 Usability Requirements

- Clear navigation for User vs Staff/Admin dashboards
- Confirmation dialogs for delete actions
- Friendly validation errors

4.0 SYSTEM ARCHITECTURE

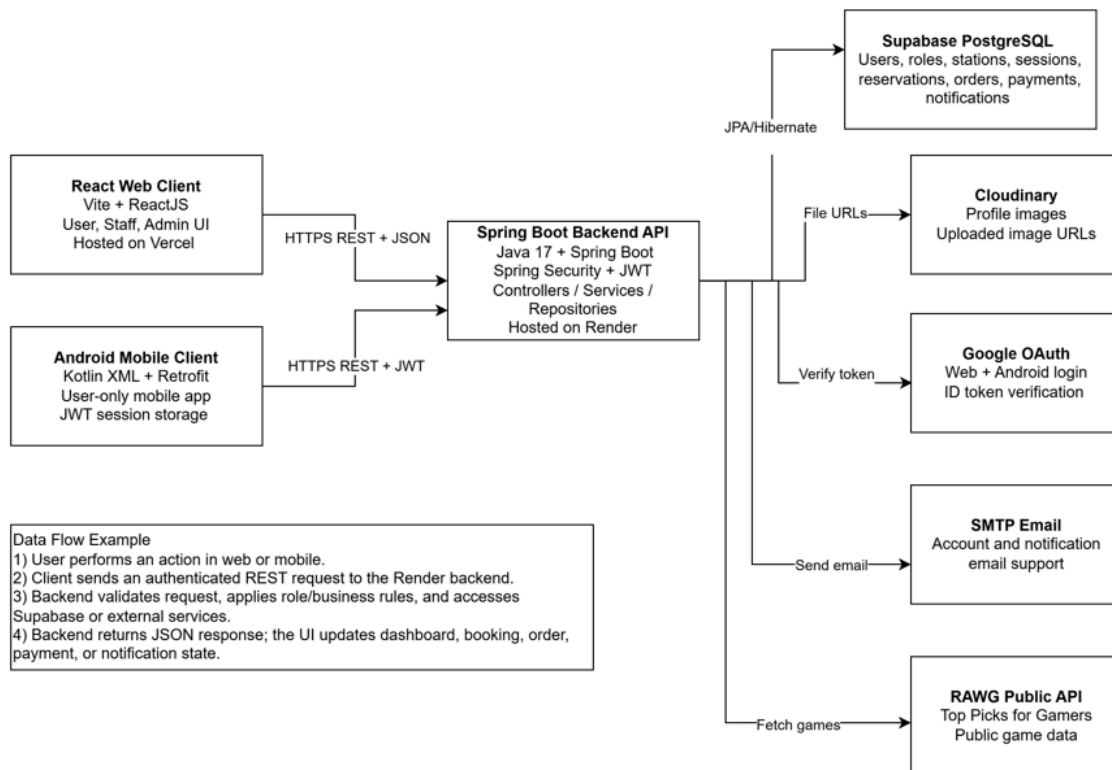
4.1 Component Diagram

Note: This should be a component diagram



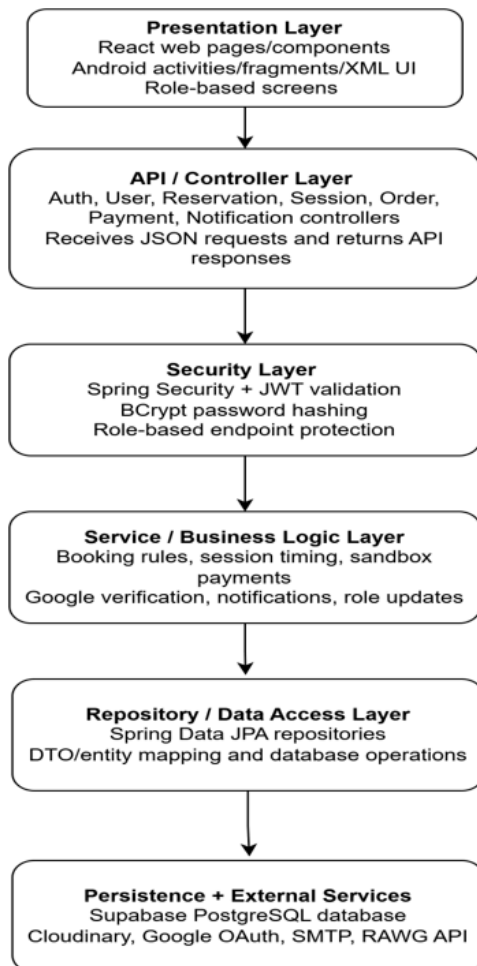
4.2 Client-Server Architecture

ByteZone uses a client-server architecture where the React web application and Android Kotlin mobile app act as clients. These clients send requests to the Spring Boot backend API. The backend processes business logic and communicates with Supabase PostgreSQL, Cloudinary, Google OAuth, SMTP, and the RAWG API. The clients do not directly access the database or external services; all communication passes through the backend API.



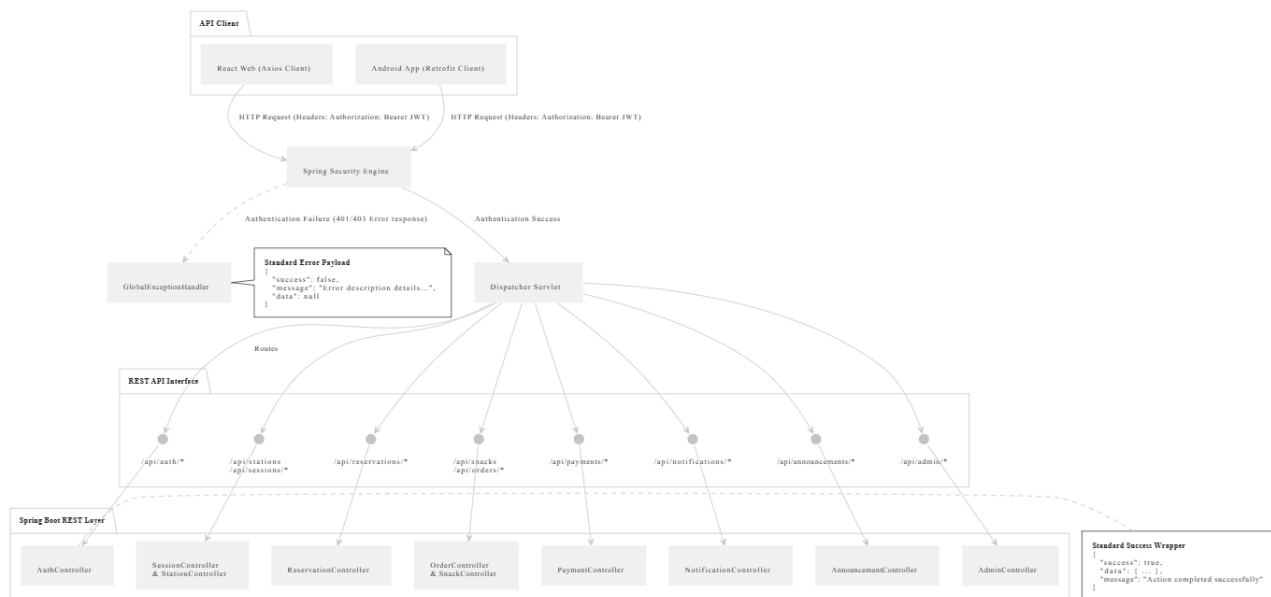
4.3 Layered Architecture

ByteZone uses a layered backend architecture to separate responsibilities. The controller layer receives requests from the web and mobile clients. The service layer handles validation and business logic such as reservation rules, session timing, payment processing, Google login, and notification handling. The repository layer communicates with the database using JPA/Hibernate. This structure makes the system easier to maintain, test, and extend.



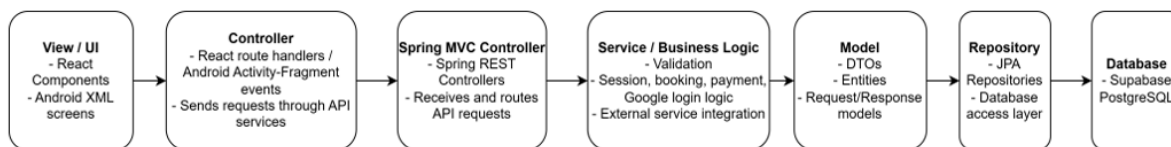
4.4 REST API Architecture

ByteZone follows a REST API architecture. The React web app and Android mobile app send HTTP requests to the Spring Boot backend using endpoints such as /api/auth, /api/reservations, /api/orders, /api/payments, and /api/admin. Protected requests pass through Spring Security and JWT authentication before reaching the correct controller. The backend processes the request and returns a standard JSON response with success, message, and data, which the web or mobile interface displays to the user.



4.5 Applicable Design Pattern

ByteZone follows an MVC-style layered pattern. The View is represented by React components and Android XML screens. User actions are handled by React route logic or Android Activity and Fragment events, then sent through API services to the Spring MVC controllers. The service layer handles validation and business logic, while the model represents DTOs, entities, and request/response objects. The repository handles database access through JPA, and the data is stored in Supabase PostgreSQL.



Technology Stack:

- **Backend:** Java 17, Spring Boot, Spring Security + JWT, Spring Data JPA, Hibernate, Maven
- **Database:** Supabase PostgreSQL
- **Web Frontend:** ReactJS, Vite, JavaScript/JSX, CSS, Axios/Fetch, Google Identity Services
- **Mobile:** Android Kotlin, XML Layouts, Retrofit, OkHttp, Gson Converter, SharedPreferences, Google Sign-In, API Level 34
- **Build Tools:** Maven (Backend), npm/Vite (Web), Gradle (Android)
- **Deployment:** Render (Backend), Vercel (Web), Android APK (Mobile)
- **External Services:** Cloudinary for profile image upload, Google OAuth for login, SMTP/Gmail App Password for email support, RAWG API for public game data
- **Security:** JWT authentication, BCrypt password hashing, protected routes, role-based access control, environment variables for secrets

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://bytezone-backend.onrender.com
API Prefix:	Most backend endpoints use the /api prefix.
Format	JSON for all requests and responses, except profile image upload which uses multipart/form-data.
Authentication	Bearer token (JWT) in Authorization header for protected endpoints.
Standard Response Structure	<pre>{ "success": true, "data": object or array or null, "message": "Request completed successfully" }</pre> Error Response Structure: <pre>{ "success": false, "data": null, "message": "Error message" }</pre>

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "fullName": "<fullName>", "email": "<email>", "password": "<password>" }</pre>
Response Structure	<pre>{ "success": true, "data": { "user": { "id": 1, "fullName": "<fullName>", "email": "<email>", "role": "USER" }, "accessToken": "<token>" }, "error": null, "timestamp": "<timestamp>" }</pre>

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": <email>, "password": <password>, }</pre>
Response Structure	<pre>{ "success": true, "data": { "user": { "id": 1, "email": "<email>", </pre>

	<pre> "role": "USER" }, "accessToken": "<token>" }, "error": null, "timestamp": "<timestamp>" } </pre>
--	--

User Logout

Description	User Logout
API URL	/auth/logout
HTTP Method	POST
Format	JSON for all requests and responses
Authentication	Required
Request Payload	<pre> { "logoutAllDevices": false } </pre>
Response Structure	<pre> { "success": true, "data": null, "error": null, "timestamp": "<timestamp>" } </pre>

Current User

Description	Get Current Authenticated User
API URL	/me
HTTP Method	GET
Format	JSON for all requests and responses
Authentication	Required
Request Payload	None
Response Structure	<pre> { "success": true, "data": { "id": 1, "email": "<email>", "role": "USER", "tournamentWins": 0 }, "error": null, "timestamp": "<timestamp>" } </pre>

	}
--	---

Google Login

Description	Google Login and Account Linking
API URL	/auth/google
HTTP Method	POST
Format	JSON for all requests and responses
Authentication	None
Request Payload	{ "googleIdToken": "<googleIdToken>" }
Response Structure	{ "success": true, "data": { "user": { "id": 1, "email": "<email>", "role": "USER" }, "accessToken": "<token>" }, "error": null, "timestamp": "<timestamp>" }

Station Endpoints

Get All Stations

Description	Get All Stations
API URL	/stations
HTTP Method	GET
Format	JSON for all requests and responses
Authentication	Required
Request Payload	None
Response Structure	{ "success": true, "data": [{ "stationId": 1, "stationNo": "PC-01", "status": "AVAILABLE" }, { "stationId": 2, "stationNo": "PC-02", "status": "IN_USE" }], }

	<pre>"error": null, "timestamp": "<timestamp>" }</pre>
--	--

Session Timer Endpoints

Start Session

Description	Start Station Session
API URL	/sessions
HTTP Method	POST
Format	JSON for all requests and responses
Authentication	Required
Request Payload	<pre>{ "stationId": 2, "userId": 1, "durationMinutes": 60 }</pre>
Response Structure	<pre>{ "success": true, "data": { "sessionId": 1001, "stationId": 2, "userId": 1, "startTime": "<timestamp>", "endTime": "<timestamp>", "status": "ACTIVE" }, "error": null, "timestamp": "<timestamp>" }</pre>

Extend Session

Description	Extend Session Time and Create Pending Transaction
API URL	/sessions/{sessionId}/extend
HTTP Method	PUT
Format	JSON for all requests and responses
Authentication	Required
Request Payload	<pre>{ "addMinutes": 30, "paymentMethod": "CASH" }</pre>
Response Structure	{

	<pre> "success": true, "data": { "sessionId": 1001, "newEndTime": "<timestamp>", "transaction": { "transactionId": 5001, "type": "SESSION_EXTENSION", "amount": 30, "status": "PENDING" } }, "error": null, "timestamp": "<timestamp>" } </pre>
--	---

End Session

Description	End Session
API URL	/sessions/{sessionId}/end
HTTP Method	PUT
Format	JSON for all requests and responses
Authentication	Required
Request Payload	None
Response Structure	<pre> { "success": true, "data": { "sessionId": 1001, "status": "ENDED" }, "error": null, "timestamp": "<timestamp>" } </pre>

Reservation Endpoints

Create Reservation

Description	Book Station Reservation
API URL	/reservations
HTTP Method	POST
Format	JSON for all requests and responses
Authentication	Required
Request Payload	<pre> { "stationId": 1, "date": "YYYY-MM-DD", </pre>

	<pre>"startTime": "HH:mm", "durationMinutes": 120 }</pre>
Response Structure	<pre>{ "success": true, "data": { "reservationId": 2001, "stationId": 1, "status": "PENDING", "date": "YYYY-MM-DD", "startTime": "HH:mm", "durationMinutes": 120 }, "error": null, "timestamp": "<timestamp>" }</pre>

Get My Reservations

Description	Get Logged-in User Reservations
API URL	/reservations/me
HTTP Method	GET
Format	JSON for all requests and responses
Authentication	Required
Request Payload	None
Response Structure	<pre>{ "success": true, "data": [{ "reservationId": 2001, "stationId": 1, "status": "PENDING" }], "error": null, "timestamp": "<timestamp>" }</pre>

Update Reservation Status

Description	Approve, Check In, Cancel Reservation
API URL	/reservations/{reservationId}
HTTP Method	PUT
Format	JSON for all requests and responses
Authentication	Required
Request Payload	<pre>{ "status": "APPROVED" }</pre>

Response Structure	<pre>{ "success": true, "data": { "reservationId": 2001, "status": "APPROVED" }, "error": null, "timestamp": "<timestamp>" }</pre>
---------------------------	--

Snack and Order Endpoints

Get Snack Menu

Description	Get Snack Items
API URL	/snacks
HTTP Method	GET
Format	JSON for all requests and responses
Authentication	Required
Request Payload	None
Response Structure	<pre>{ "success": true, "data": [{ "snackId": 1, "name": "Chips", "price": 25, "available": true }], "error": null, "timestamp": "<timestamp>" }</pre>

Place Order

Description	Create Snack Order
API URL	/orders
HTTP Method	POST
Format	JSON for all requests and responses
Authentication	Required
Request Payload	<pre>{ "stationId": 2, "items": [{ "snackId": 1, "qty": 2 }], "paymentMethod": "QR" }</pre>
Response Structure	<pre>{ "success": true, "data": { "orderId": 3001, </pre>

	<pre> "status": "PENDING", "total": 50 }, "error": null, "timestamp": "<timestamp>" } </pre>
--	--

Update Order Status

Description	Update Snack Order Status
API URL	/orders/{orderId}/status
HTTP Method	PUT
Format	JSON for all requests and responses
Authentication	Required
Request Payload	<pre> { "status": "READY" } </pre>
Response Structure	<pre> { "success": true, "data": { "orderId": 3001, "status": "READY" }, "error": null, "timestamp": "<timestamp>" } </pre>

Payment and Transaction Endpoints

Get Pending Payments

Description	View Pending Payments List
API URL	/payments/pending
HTTP Method	GET
Format	JSON for all requests and responses
Authentication	Required
Request Payload	None
Response Structure	<pre> { "success": true, "data": [{ "paymentId": 9001, "type": "SESSION_EXTENSION", "amount": 30, "status": "PENDING" }], "error": null, "timestamp": "<timestamp>" } </pre>

Confirm Payment

Description	Confirm Pending Payment
API URL	/payments/{paymentId}/confirm
HTTP Method	PUT
Format	JSON for all requests and responses
Authentication	Required
Request Payload	<pre>{ "referenceNo": "<referenceNo>", "method": "CASH" }</pre>
Response Structure	<pre>{ "success": true, "data": { "paymentId": 9001, "status": "PAID" }, "error": null, "timestamp": "<timestamp>" }</pre>

Transaction History

Description	View Transaction History
API URL	/payments
HTTP Method	GET
Format	JSON for all requests and responses
Authentication	Required
Request Payload	None
Response Structure	<pre>{ "success": true, "data": [{ "transactionId": 5001, "type": "SESSION_EXTENSION", "amount": 30, "status": "PAID" }, { "transactionId": 5002, "type": "SNACK_ORDER", "amount": 50, "status": "PENDING" }], "error": null, "timestamp": "<timestamp>" }</pre>

Admin Endpoints

Admin Metrics

Description	View Dashboard Metrics
API URL	/admin/metrics
HTTP Method	GET
Format	JSON for all requests and responses
Authentication	Required
Request Payload	None
Response Structure	<pre>{ "success": true, "data": { "totalUsers": 120, "newUsers": 5, "activeUsers": 18, "totalTransactions": 300, "pendingPayments": 7 }, "error": null, "timestamp": "<timestamp>" }</pre>

Update User Role and Tournament Wins

Description	Update User Role and Tournament Wins
API URL	/admin/users/{userId}
HTTP Method	PUT
Format	JSON for all requests and responses
Authentication	Required
Request Payload	<pre>{ "role": "STAFF", "tournamentWins": 3 }</pre>
Response Structure	<pre>{ "success": true, "data": { "userId": 1, "role": "STAFF", "tournamentWins": 3 }, "error": null, "timestamp": "<timestamp>" }</pre>

Announcements Endpoints

Create Announcement

Description	Create Announcement
API URL	/announcements
HTTP Method	POST
Format	JSON for all requests and responses
Authentication	Required
Request Payload	<pre>{ "title": "<title>", "content": "<content>" }</pre>
Response Structure	<pre>{ "success": true, "data": { "announcementId": 8001, "title": "<title>" }, "error": null, "timestamp": "<timestamp>" }</pre>

File Upload Endpoints

Upload File

Description	Upload File and Link to Record
API URL	/files/upload
HTTP Method	POST
Format	Multipart form data
Authentication	Required
Request Payload	Form-data - file: upload file - recordType: PAYMENT or ORDER or RESERVATION - recordId: number
Response Structure	<pre>{ "success": true, "data": { "fileId": 7001, "fileName": "<fileName>", "recordType": "<recordType>", "recordId": 9001 }, "error": null, }</pre>

	"timestamp": "<timestamp>" }
--	---------------------------------

Additional Implemented Endpoints:

Profile Image Upload

API URL	/api/user/me/profile-image
Description:	Uploads the authenticated user's profile image to Cloudinary and saves the image URL in the user record.
HTTP Method	POST
Format	multipart/form-data
Authentication	Required
Request Payload	Form-data: file: <image_file>
Response Structure	{ "success": true, "data": { "id": 1, "fullName": "<fullName>", "email": "<email>", "role": "USER", "profileImageUrl": "<uploaded_image_url>" }, "message": "Profile image uploaded successfully" }
Notes:	The uploaded image is stored through Cloudinary, and the returned image URL is saved in the authenticated user's profile record.

Public Gaming Highlights

API URL	/api/public/gaming-highlights
Description:	Retrieve public game highlights for Top Picks for Gamers
HTTP Method	GET
Format	JSON
Authentication	Not Required
Request Payload	None
Response Structure	<pre>{ "success": true, "data": [{ "name": "<game_name>", "background": "<image_url>", "rating": 4.5, "released": "YYYY-MM-DD" }], "message": "Gaming highlights fetched successfully" }</pre>
Notes:	This endpoint retrieves public game data from the RAWG API and displays it in the Top Picks for Gamers section.

Sandbox Payment Result

API URL	/api/payments/sandbox/{paymentId}/success /api/payments/sandbox/{paymentId}/fail /api/payments/sandbox/{paymentId}/cancel
Description:	Updates the result/status of a sandbox payment transaction
HTTP Method	POST
Format	JSON
Authentication	Required
Request Payload	None
Response Structure	<pre>{ "success": true, "data": { "paymentId": 9001, "type": "SNACK_ORDER", "amount": 50.00, } }</pre>

	<pre> "status": "PAID FAILED CANCELLED", "method": "SANDBOX", "referenceNo": "BZ-SANDBOX-9001" }, "message": "Sandbox payment result updated successfully" } </pre>
Notes:	Payment types may include RESERVATION, SNACK_ORDER, and SESSION_EXTENSION. Payment status depends on the selected sandbox result action: PAID, FAILED, or CANCELLED.

5.3 Error Handling

HTTP Status Codes

- 200 OK Successful request
- 201 Created Resource created
- 400 Bad Request Invalid input
- 401 Unauthorized Authentication required or failed
- 403 Forbidden Insufficient permissions
- 404 Not Found Resource does not exist
- 409 Conflict Duplicate or rule conflict
- 500 Internal Server Error Server error

Error Code Examples

Example 1	<pre> { "success": false, "data": null, "error": { "code": "BUSINESS-002", "message": "Station not available", "details": "Selected station is already reserved or currently in use." }, "timestamp": "<timestamp>" } </pre>
Example 2	<pre> { "success": false, </pre>

	<pre> "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "email": "Email format is invalid", "password": "Password must be at least 8 characters" } }, "timestamp": "<timestamp>" } </pre>
--	--

Common Error Codes

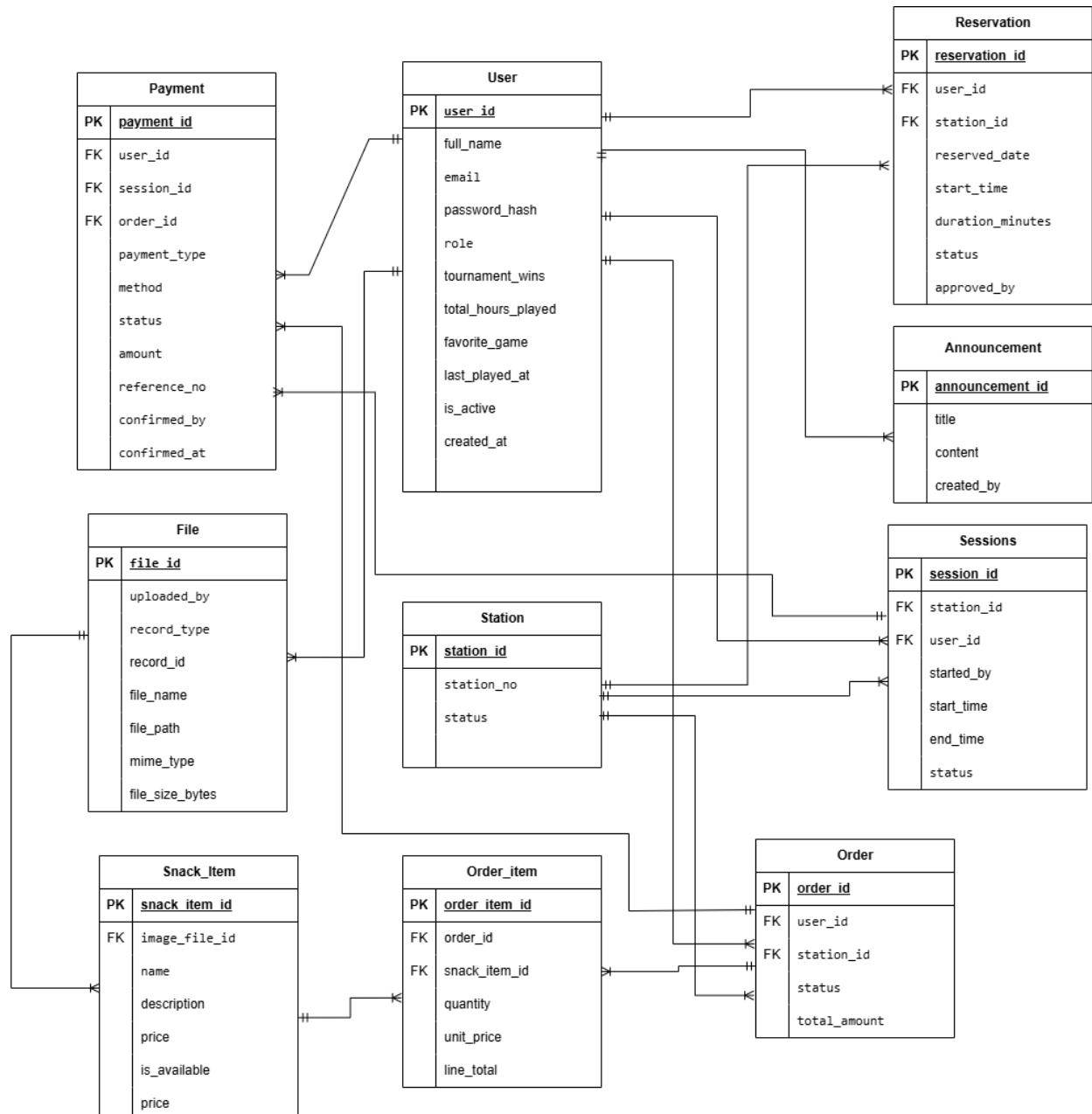
- AUTH-001 Invalid credentials
- AUTH-002 Token expired
- AUTH-003 Insufficient permissions
- VALID-001 Validation failed
- DB-001 Resource not found
- DB-002 Duplicate entry
- BUSINESS-001 Reservation rule violated
- BUSINESS-002 Station not available
- ORDER-001 Item unavailable
- ORDER-002 Empty order
- PAYMENT-001 Payment not found
- PAYMENT-002 Payment already confirmed
- SYSTEM-001 Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)

The final implementation uses Supabase PostgreSQL as the hosted relational database. The database design still follows the ERD structure shown below, but the deployed database is hosted in Supabase instead of a local MySQL database.



Detailed Relationships:

- **One-to-Many:** User → Session
- **Many-to-One:** Session → Station
- **One-to-Many:** Users → Reservations
- **Many-to-One:** Reservation → Station
- **One-to-Many:** User → Order
- **Many-to-One:** Order → Station
- **One-to-Many:** Order → OrderItem
- **Many-to-One:** OrderItem → SnackItem
- **One-to-Many:** User → Payment
- **Many-to-One:** Payment → Session
- **Many-to-One:** Payments → Orders
- **One-to-Many:** User → Announcement
- **One-to-Many:** User → File
- **Many-to-One:** SnackItem → File

Key Tables:

1. **user** - user accounts, roles, and player statistics
2. **station** - station records and availability status
3. **session** - station timer sessions and session history
4. **reservation** - station booking records and reservation statuses
5. **snack_item** - snack menu items and availability
6. **order** - snack orders placed by users
7. **order_item** - items inside each snack order
8. **payment** - pending and paid records for session extensions and snack orders
9. **announcement** - system announcements shown to users
10. **file / profile image record** - uploaded profile image information or image URL linked to user profile records
11. **notification** - user notification records for reservation, payment, and order updates

Table Structure Summary:

- **user:** user_id, full_name, email, password_hash, role, tournament_wins, total_hours_played, favorite_game, last_played_at, is_active
- **station:** station_id, station_no, status
- **session:** session_id, station_id, user_id, started_by, start_time, end_time, status

- **reservation:** reservation_id, user_id, station_id, reserved_date, start_time, duration_minutes, status, approved_by
- **snack_item:** snack_item_id, name, description, price, is_available, image_file_id
- **order:** order_id, user_id, station_id, status, total_amount
- **order_item:** order_item_id, order_id, snack_item_id, quantity, unit_price, line_total
- **payment:** payment_id, user_id, payment_type, session_id, order_id, method, status, amount, reference_no, confirmed_by, confirmed_at
- **announcement:** announcement_id, title, content, created_by
- **file:** file_id, uploaded_by, record_type, record_id, file_name, file_path, mime_type, file_size_bytes

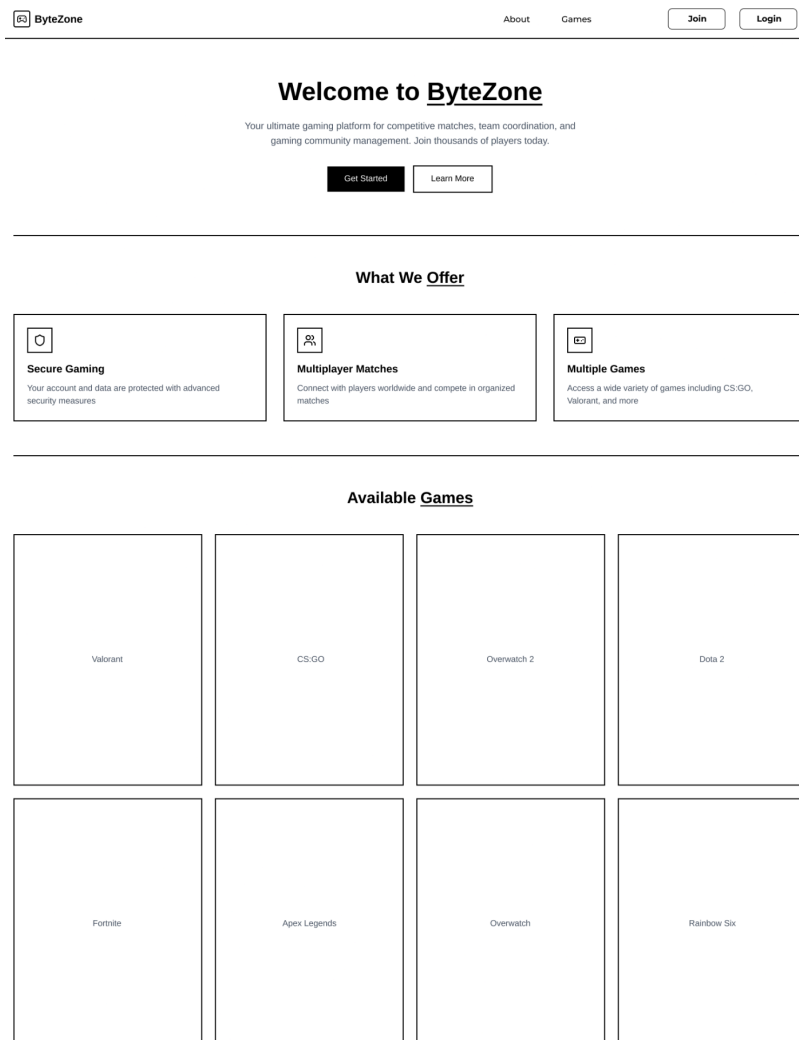
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

A. Public and Authentication Screens

Landing Page



Register Page



[About](#) [Games](#) [Join](#) [Login](#)



Welcome back
Please enter your details to sign in

Email

 Enter your email

 Enter your password



Forgot Password

Sign in

Or sign in with



Don't have an account? **Click here**

© ByteZone

[About](#) [Games](#) [Contact](#)

Login Page



[About](#) [Games](#) [Join](#) [Login](#)

Create your account
Please enter your details to sign up

Fullname

 Enter your fullname

Email

 Enter your email

Password

 Enter your password



 Confirm your password



Create account

Or sign in with



Already have an account? **Click here**

© ByteZone

[About](#) [Games](#) [Contact](#)

User Dashboard

HomeBookOrder

Welcome, Lichael

Logout

Like's Status

Total Hours Played

47

Tournament Won

12

Last Played

N/A

Favorite Game

N/A

Current Session

Remaining Time

00:15:00

Current Game

Valorant

Station

S4

Extend Playtime

+ Add Time

Reservation Status

Date and Time

February 14, 2026
01:33 PM - 02:38PM

Reserved Station

S8

Hours Remaining

01:05:00

Session Status

Inactive

Lichael

Member

Member Since: Jan 2023
Last Visit: 2 days ago

Quick Actions

Book Station

Order Snacks

View Tournaments

ByteZone Updates

Valorant Tournament

Today, 8:00 PM
Prize Pool: \$500

League Night

Tomorrow, 7:00 PM
Free Entry

CS:GO Championship

This Weekend
Prize Pool: \$1000

Top Picks for Gamers

<

Valorant

League of Legends

Counter-Strike:
Global Offensive (CS:GO)

Minecraft

Fortnite

>

© ByteZone

HomeBookOrder

42

Account Information Page

[Home](#)

[Book](#)

[Order](#)

 **Welcome, Lichael** 



Account Information



120 × 120

Click the icon to upload a new profile picture

Full Name

Email Address

Book Station Page

[Home](#)

[Book](#)

[Order](#)

Welcome, Lichael  



Book a GAMING Session

Select your GAMING station

☐

Available

☐

Unavailable

☐

Selected

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

☐

Select DATE & TIME

>

>

Summary

Station	Not Selected
Date	Not Selected
Time	Not Selected
Total	P 0

© ByteZone

[Home](#) [Book](#) [Order](#)

Snack Ordering Page

HomeBookOrder

Welcome, Lichael

Logout

←

Grab a SNACK

Recommended offers

Spicy Ramen Bowl

P120

Classic Burger

P150

Chicken Wings

P180

Pizza Slice

P95

Noodles & Soups

Beef Noodle Soup

P140

Chicken Ramen

P135

Miso Soup

P90

Seafood Noodles

P165

Drinks & Beverages

Iced Coffee

P65

Bubble Tea

P85

Fresh Orange Juice

P70

Smoothie Bowl

P95

Summary

Station

Not Selected

Total

P 0

Confirm

Cancel

© ByteZone

HomeBookOrderContact

Admin/Staff Dashboard Page

☐ ADMIN DASHBOARD

Welcome, Lichael

Logout

☐ Dashboard

☐ User Management

☐ Announcements

☐ Extending Hours

☐ Snacks

☐ Orders

☐ Transaction History

☐ Pending Payments

TOTAL USERS

NEW USERS

ACTIVE USERS

ANNOUNCEMENTS

TRANSACTIONS

Recent Users

NAME	EMAIL	STATUS	PLAYING	ROLE	REGISTRATION DATE
------	-------	--------	---------	------	-------------------

Recent Announcements

Orders

Recent Transactions

44

Admin/Staff User Management Page

☐ ADMIN DASHBOARD

Welcome, Lichael

Logout

- ☐ Dashboard
- ☐ User Management
- ☐ Announcements
- ☐ Extending Hours
- ☐ Orders
- ☐ Transaction History
- ☐ Pending Payments

User Management

User List								search users
ID	NAME	EMAIL	STATUS	PLAYING	ROLE	REGISTRATION DATE	LAST ACTIVITY	ACTIONS
								Edit Delete

Admin/Staff Announcements Page

☐ ADMIN DASHBOARD

Welcome, Lichael

Logout

- ☐ Dashboard
- ☐ User Management
- ☐ Announcements
- ☐ Extending Hours
- ☐ Orders
- ☐ Transaction History
- ☐ Pending Payments

Announcements

Announcements List						search users
ID	TITLE	DESCRIPTION	CREATED BY	CREATED AT	ACTIONS	
						Edit Delete

Admin/Staff Orders Page

☐ ADMIN DASHBOARD

Welcome, Lichael

Logout

- ☐ Dashboard
- ☐ User Management
- ☐ Announcements
- ☐ Extending Hours
- ☐ Orders
- ☐ Transaction History
- ☐ Pending Payments

Orders

Orders List						☐ search users
ID	NAME	QUANTITY	TOTAL	ORDER DATE	ACTIONS	
					Edit	Done

Admin/Staff Snacks Page

☐ ADMIN DASHBOARD

Welcome, Lichael

Logout

- ☐ Dashboard
- ☐ User Management
- ☐ Announcements
- ☐ Extending Hours
- ☐ Snacks
- ☐ Orders
- ☐ Transaction History
- ☐ Pending Payments

Snacks

Add new food

Snacks List							☐ search users
ID	IMAGE	NAME	PRICE	CATEGORY	CREATED AT	ACTIONS	
						Edit	Delete

Admin/Staff Transaction History Page

☐ ADMIN DASHBOARD

Welcome, Lichael

- ☐ Dashboard
- ☐ User Management
- ☐ Announcements
- ☐ Extending Hours
 - Snacks
- ☐ Orders
- ☐ Transaction History
- ☐ Pending Payments

Transaction History

Transaction List						
☐ search users						
ID	USER	TYPE	AMOUNT	STATUS	DESCRIPTION	DATE

Admin/Staff Extending Hours Page

☐ ADMIN DASHBOARD

Welcome, Lichael

Logout

- ☐ Dashboard
- ☐ User Management
- ☐ Announcements
- ☐ Extending Time
 - Snacks
- ☐ Orders
- ☐ Transaction History
- ☐ Pending Payments

Extending Time

☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

☐ Available ☐ Unavailable ☐ Selected

Enter time	Hour
Confirm	Cancel

Admin/Staff Pending Payments Page

☐ ADMIN DASHBOARD

Welcome, Lichael Logout

☐ Dashboard

☐ User Management

☐ Announcements

☐ Extending Hours

☐ Snacks

☐ Orders

☐ Transaction History

☐ Pending Payments

Pending Payments

Pending List

ID	NAME	TYPE	AMOUNT	STATUS	DESCRIPTION	DATE	ACTION
----	------	------	--------	--------	-------------	------	--------

7.2 Mobile Application Wireframes

Login Page

11:11

←

CREATE YOUR NEW ACCOUNT

Enter full name

Enter email

Enter password

Confirm password

☐ By creating account you agree to our terms of conditions.

Create account

Register Page

11:11

ByteZone

LOGIN TO YOUR ACCOUNT

Email

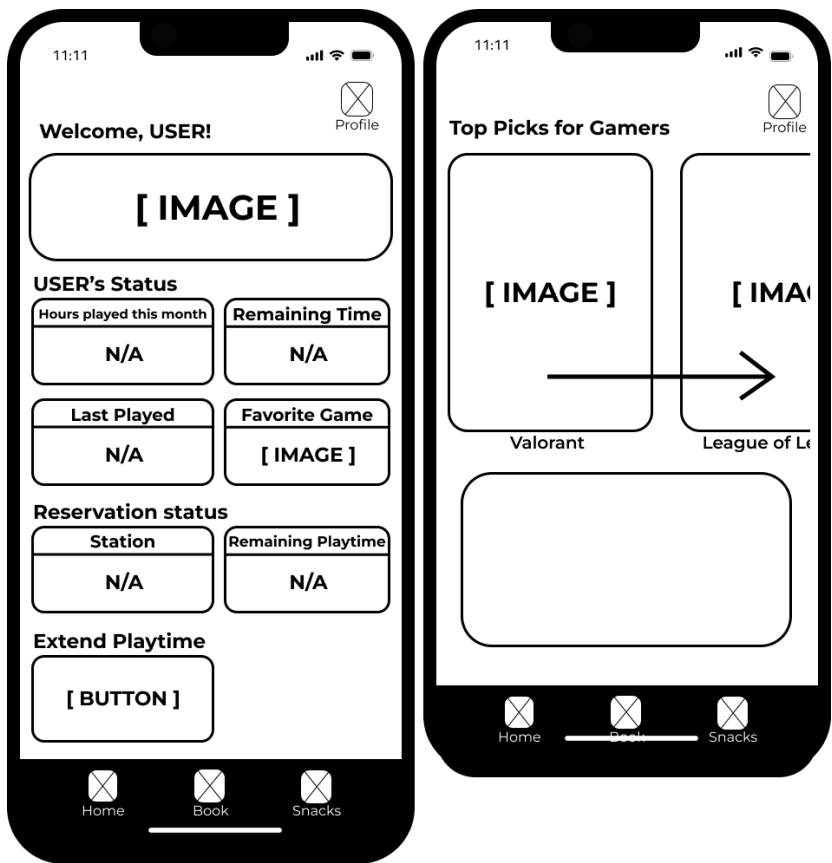
Password

Forgot Password

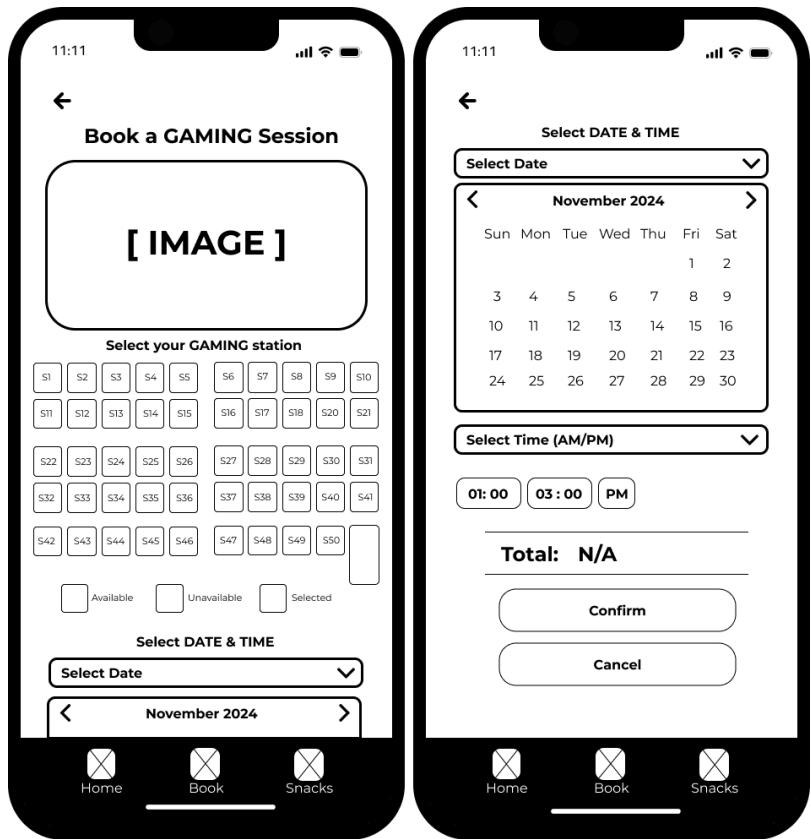
Sign in

Create new account

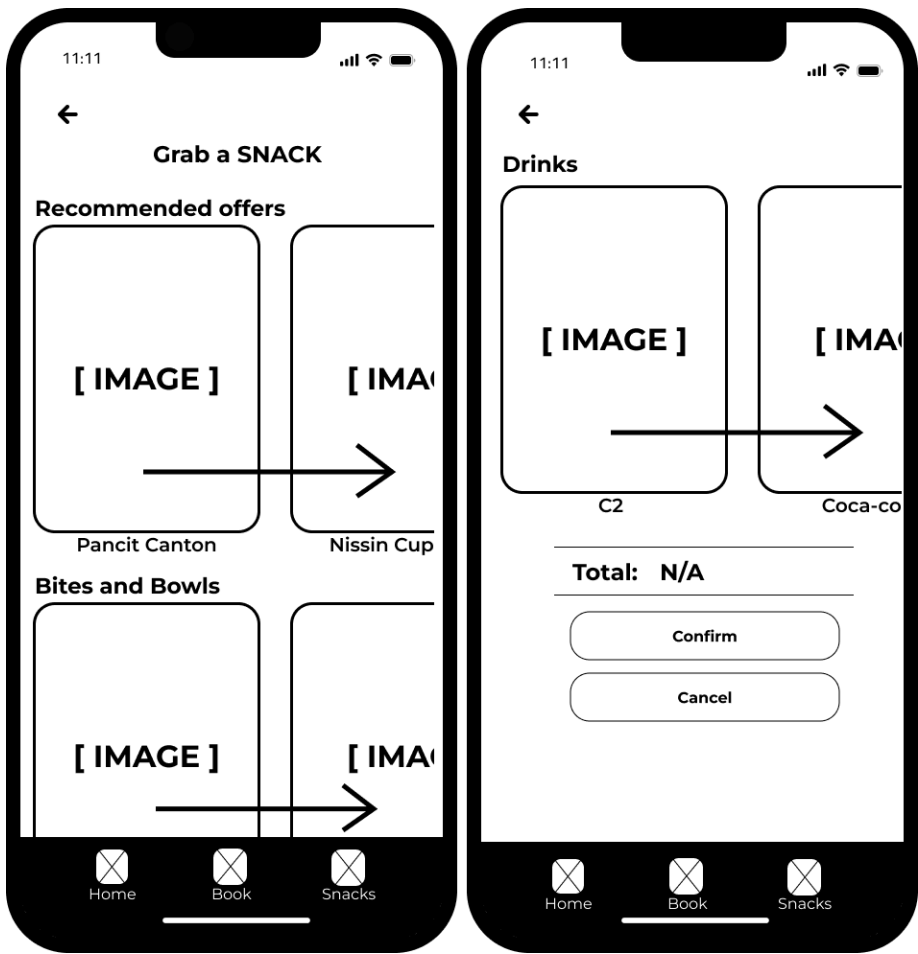
User Dashboard page



Book Station Page



Snack Ordering Page



8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1–2: Confirm scope, roles, and feature list for User, Staff, and Admin

Day 3–4: Complete Functional Requirements and Non-Functional Requirements

Day 5–7: Finalize system architecture and module breakdown

Week 2: Detailed Design

Day 1–2: Finalize API endpoint list and access control per role

Day 3–4: Finalize database tables and relationships

Day 5–6: Finalize UI and UX wireframes for web and mobile

Day 7: Finalize implementation plan and testing plan

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with required dependencies and base project structure

Day 2: Database configuration, entities, and repositories

Day 3: Authentication implementation and protected access

Day 4: Role-based access control for User, Staff, and Admin

Day 5–7: User management endpoints and core station endpoints

Week 4: Core Features

Day 1: Station session timer module start and end session

Day 2: Extend session module and transaction creation

Day 3: Reservation module with one booking per day rule and next day booking

Day 4: Snack items and snack ordering module with order status flow

Day 5: Announcements module and transaction history module

Day 6–7: Validation, error handling, and backend testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React project setup, routing, and layout

Day 2: Authentication pages and protected route handling

Day 3: User dashboard main sections and navigation

Day 4: Book station page and reservation status section

Day 5–7: Snack menu, cart and checkout, order status, and order history pages

Week 6: Complete Web Features

Day 1: Staff dashboard with active sessions and pending payments section

Day 2: Extending hours page with pending and paid handling

Day 3: Orders management page with status updates from pending to served

Day 4: Admin pages user management role assignment and tournament wins update

Day 5: Announcements and transaction history pages for staff and admin

Day 6–7: Full API integration testing and UI polish

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup, project structure, Retrofit base setup

Day 2: Authentication screens and token storage

Day 3: Mobile dashboard with current session and reservation status

Day 4: Book station flow and reservation viewing

Day 5–7: Snack menu, simple checkout, and order status screens

Week 8: Complete Mobile App

Day 1: Extend playtime request screen and transaction viewing

Day 2: Transaction history screen and profile screen

Day 3: Notifications panel for order updates and payment reminders

Day 4: Testing on emulator and device and bug fixing

Day 5: APK generation and final mobile documentation updates

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: Public API integration for Top Picks for Gamers

Day 2: Google login integration and account linking

Day 3: File upload integration linked to records with view and download

Day 4: Email sending for account and notification purposes

Day 5–7: End-to-end testing across backend, web, and mobile and bug fixing

Week 10: Deployment

Day 1: Backend deployment and environment configuration

Day 2: Web application deployment

Day 3: Mobile APK distribution and final checks

Day 4: Final system testing and performance checks

Day 5: Final document review and project submission

Phase 6: Final Integration, Deployment, and Testing

- Connected backend to Supabase PostgreSQL
- Deployed Spring Boot backend to Render using Docker
- Deployed React/Vite web frontend to Vercel
- Configured Vercel environment variables for backend API URL and Google Client ID
- Configured Render environment variables for database, JWT, Cloudinary, SMTP, Google OAuth, and RAWG API
- Completed Android Kotlin mobile app connected to deployed backend through Retrofit
- Configured Android Google OAuth using package name, SHA-1, SHA-256, and Web Client ID
- Tested user registration, login, Google login, role-based access, booking, snack ordering, sandbox payment, notifications, transaction history, session timing, profile update, and profile image upload

Milestones:

- **M1 End Week 2:** SDD complete with final requirements, database, API list, and wireframes
- **M2 End Week 4:** Backend complete with authentication, roles, and core modules
- **M3 End Week 6:** Web application complete for User, Staff, and Admin
- **M4 End Week 8:** Mobile application complete and connected to live backend
- **M5 End Week 10:** Integrations completed, system deployed, and final submission ready

Critical Path:

1. Authentication and protected access implementation in Week 3
2. Role-based access control implementation in Week 3
3. Station session timer module in Week 4
4. Reservation module with booking rules in Week 4
5. Snack ordering module with full status flow in Week 4
6. Full web integration testing in Week 6
7. Cross-platform end-to-end testing in Week 9
8. Deployment and final validation in Week 10

Risk Mitigation:

- Start with the simplest working version of each core feature before adding enhancements
- Freeze the scope after Week 4 to avoid last-minute feature additions
- Test role restrictions early to prevent security issues later
- Enforce booking rules and station availability checks at the backend to avoid conflicts
- Force order status to follow the correct flow to prevent skipping steps

- Keep regular backups and working versions before major changes
- Start integrations early in Week 9 and test each integration independently before combining them